

StringWriter and StringReader in C# with Examples

In this article, I am going to discuss **StringWriter and StringReader in C#** with Examples. Please read our previous article where we discussed [BinaryWriter and BinaryReader in C#](#) with Examples. At the end of this article, you will understand what StringWriter and StringReader are in C# and when and how to use StringWriter and StringReader in C# with Examples.

What is StringWriter Class in C#?

The StringWriter Class in C# is derived from the TextWriter class and this StringWriter class is mainly used to manipulate strings rather than files. The StringWriter class enables us to write a string either synchronously or asynchronously. So, we can write a character either with Write(Char) or WriteAsync(Char) method and we can write a string with Write(String) or WriteAsync(String) method. The text or information written by StringWriter class is stored inside a StringBuilder. The Text namespace and the strings can be built efficiently using the StringBuilder class because strings are immutable in C# and the Write and WriteLine methods provided by StringWriter help us to write into the object of StringBuilder. StringBuilder class stores the information written by StringWriter class.

The most important point that you need to remember is that the StringWriter is not for writing files on the local disk. It is used for manipulating strings and it saves information in StringBuilder. If you go to the definition of StringWriter class, then you will see the following.

```
namespace System.IO
{
    ...public class StringWriter : TextWriter
    {
        ...public StringWriter();
        ...public StringWriter(IFormatProvider formatProvider);
        ...public StringWriter(StringBuilder sb);
        ...public StringWriter(StringBuilder sb, IFormatProvider formatProvider);

        ...public override Encoding Encoding { get; }

        ...public override void Close();
        ...public override Task FlushAsync();
        ...public virtual StringBuilder GetStringBuilder();
        ...public override string ToString();
        ...public override void Write(char value);
        ...public override void Write(char[] buffer, int index, int count);
        ...public override void Write(string value);
        ...public override Task WriteAsync(char value);
        ...public override Task WriteAsync(string value);
        ...public override Task WriteAsync(char[] buffer, int index, int count);
        ...public override Task WriteLineAsync(char value);
        ...public override Task WriteLineAsync(string value);
        ...public override Task WriteLineAsync(char[] buffer, int index, int count);
        ...protected override void Dispose(bool disposing);
    }
}
```

Let us understand the constructors, properties, and methods of StringWriter class in C#.

Constructors of StringWriter in C#

1. **public StringWriter():** It initializes a new instance of the System.IO.StringWriter class.
2. **public StringWriter(IFormatProvider formatProvider):** It initializes a new instance of the StringWriter class with the specified format control. The parameter formatProvider specifies a System.IFormatProvider object that controls formatting
3. **public StringWriter(StringBuilder sb):** It initializes a new instance of the StringWriter class that writes to the specified System.Text.StringBuilder. The parameter sb specifies the StringBuilder object to write to.
4. **public StringWriter(StringBuilder sb, IFormatProvider formatProvider):** It initializes a new instance of the StringWriter class that writes to the specified StringBuilder and has the specified format provider. The parameter sb specifies the StringBuilder object to write to and the parameter formatProvider specifies a System.IFormatProvider object that controls formatting.

Properties of StringWriter Class in C#:

The StringWriter class in C# provides the following property.

1. **Encoding:** This property is used to get the Encoding in which the output is written. So, it returns the Encoding in which the output is written.

Methods of StringWriter Class in C#

The StringWriter Class in C# provides the following methods.

1. **Close():** This method is used to close the current StringWriter and the underlying stream.
2. **Dispose():** This method is used to release the unmanaged resources used by the System.IO.StringWriter and optionally releases the managed resources.
3. **FlushAsync():** This method is used to asynchronously clear all buffers for the current writer and causes any buffered data to be written to the underlying device.
4. **GetStringBuilder():** This method is used to return the underlying StringBuilder.
5. **ToString():** This method is used to return a string containing the characters written to the current StringWriter so far.
6. **Write(char value):** This method is used to write a character to the string.
7. **Write(string value):** This method is used to write a string to the current string.
8. **WriteAsync(char value):** This method is used to write a character to the string asynchronously.
9. **WriteAsync(string value):** This method is used to write a string to the current string asynchronously.
10. **WriteLine(String):** This method is used to Write a string followed by a line terminator to the text string or stream.
11. **WriteLineAsync(string value):** This method is used to write a string followed by a line terminator asynchronously to the current string.

Example to Understand StringWriter Class in C#:

In the below example, we use StringWriter and StringReader Class in C#. Here, the string variable text stores a string value, and using StringWriter, we store this string value in StringBuilder. Then, using StringReader we get the data and displayed the data on the console.

```

using System;
using System.Text;
using System.IO;
namespace StringWriter_StreamReader_Demo
{
    class Program
    {
        static void Main(string[] args)
        {
            string text = "Hello. This is First Line \nHello. This is Second L
\nHello. This is Third Line";
            //Writing string into StringBuilder
            StringBuilder stringBuilder = new StringBuilder();
            StringWriter stringWriter = new StringWriter(stringBuilder);
            //Store Data on StringBuilder
            stringWriter.WriteLine(text);
            stringWriter.Flush();
            stringWriter.Close();
            //Read Entry
            StreamReader reader = new StreamReader(stringBuilder.ToString());
            //Check to End of File
            while (reader.Peek() > -1)
            {
                Console.WriteLine(reader.ReadLine());
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

Hello. This is First Line
Hello. This is Second Line
Hello. This is Third Line

```

Note: Being a derived class of the TextWriter class, the StringWriter class is used for writing and dealing with the string data instead of the files with the purpose to manipulate the string data and storing the result into the StringBuilder.

What is StreamReader Class in C#?

The StreamReader class in C# is derived from the TextReader class and this StreamReader class is mainly used to manipulate strings rather than files. This StreamReader class is built using a string and Read and ReadLine methods are provided by StreamReader class to read the parts of the string. The data read by the StreamReader class is the data written by the StringWriter class which is derived from the TextWriter subclass. The data can be read in a synchronous manner or in an asynchronous manner using StreamReader class and the read operations are performed by using the constructors and methods present in the StreamReader Class.

So, in simple words, we can say that the `StringReader` class in C# implements `TextReader` class that reads a string from a string. It enables you to read a string synchronously or asynchronously. You can read a character with `Read()` method and read a line with `ReadLine()` method. If you go to the definition of `StringReader` class, then you will see the following.

```
namespace System.IO
{
    ...public class StringReader : TextReader
    {
        ...public StringReader(string s);

        ...public override void Close();
        ...public override int Peek();
        ...public override int Read();
        ...public override int Read(char[] buffer, int index, int count);
        ...public override Task<int> ReadAsync(char[] buffer, int index, int count);
        ...public override Task<int> ReadBlockAsync(char[] buffer, int index, int count);
        ...public override string ReadLine();
        ...public override Task<string> ReadLineAsync();
        ...public override string ReadToEnd();
        ...public override Task<string> ReadToEndAsync();
        ...protected override void Dispose(bool disposing);
    }
}
```

Let us understand the constructors, and methods of `StringReader` class in C#.

The Constructor of StringReader Class in C#

public StringReader(string s): It initializes a new instance of the `StringReader` class that reads from the specified string. Here, the parameter “s” specifies the string to which the `StringReader` should be initialized.

Methods of StringReader Class in C#

The `StringReader` class in C# provides the following methods.

1. **Close():** This method is used to close the `StringReader`.
2. **Peek():** This method is used to return the next available character but does not consume it. It returns an integer representing the next character to be read, or -1 if no more characters are available or the stream does not support seeking.
3. **Read():** This method is used to read the next character from the input string and advances the character position by one character. It returns the next character from the underlying string, or -1 if no more characters are available.
4. **ReadLine():** This method is used to read a line of characters from the current string and returns the data as a string. It returns the next line from the current string, or null if the end of the string is reached.
5. **ReadLineAsync():** This method is used to read a line of characters asynchronously from the current string and returns the data as a string. It returns a task that represents the asynchronous read operation. The value of the `TaskResult` parameter contains the next line from the string reader or is null if all the characters have been read.

6. **ReadToEnd():** This method is used to read all characters from the current position to the end of the string and returns them as a single string. It returns the content from the current position to the end of the underlying string.
7. **ReadToEndAsync():** This method is used to read all characters from the current position to the end of the string asynchronously and returns them as a single string. It returns a task that represents the asynchronous read operation. The value of the TResult parameter contains a string with the characters from the current position to the end of the string.
8. **Dispose():** This method is used to release the unmanaged resources used by the System.IO.StringReader and optionally releases the managed resources.

Example to Understand StringReader Class in C#:

In the below example, the StringReader class reads a string from a string variable and marks each line with the count number, and then displays it on the console.

```
using System;
using System.IO;
namespace StringWriter_StringReader_Demo
{
    class Program
    {
        static void Main(string[] args)
        {
            string text = @"You are reading
                           this StringWriter and StringReader in C# article :
                           dotnettutorials.net";
            using (StringReader rstringReader = new StringReader(text))
            {
                int count = 0;
                string line;
                while ((line = rstringReader.ReadLine()) != null)
                {
                    count++;
                    Console.WriteLine("Line {0}: {1}", count, line);
                }
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Line 1: You are reading
Line 2:                this StringWriter and StringReader in C# article at
Line 3:                dotnettutorials.net
```

In the next article, I am going to discuss [FileInfo Class in C#](#) with Examples. Here, in this article, I try to explain **StringWriter and StringReader in C#** with Examples. I hope you enjoy this **StringWriter and**

StringReader in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.